

INTERNATIONAL CONFERENCE ON RECENT TRENDS IN ADVANCED COMPUTING
2019, ICRTAC 2019Implementing smart swapping algorithm to boost the performance
of Linux PC's.Shivam Sharma^{*}, S.A. Sajidha*Vellore Institute of Technology, Chennai, 600127, India*

Abstract

Swap space in Linux is used when the amount of physical memory (RAM) is full. If the system needs more memory resources and the RAM is full, inactive pages in memory are moved to the swap space. While swap space can help machines with a small amount of RAM, it should not be considered a replacement for more RAM. Swap space is located on hard drives, which have a slower access time than physical memory. If any memory-intensive application is deployed on a PC with limited amount of Random-Access Memory, the inactive (not recently used) application which is generally the system UI is shifted to the Swap memory (much slower than RAM). This makes the entire UI slow but the higher priority processes (the most recent applications) remain unaffected because they remain in the RAM. When we are finished with our work with the memory intensive application and close it, the RAM gets cleared and has ample of space to accommodate the contents of the SWAP memory. But the swap memory stays as it is, and does not get transferred back to the RAM. This results in slower UI even though the system has enough RAM and no memory intensive process is running. This then results in lag in UI transition, delayed application launches, slower processing of programs and a large overhead. In years past, the recommended amount of swap space increased linearly with the amount of RAM in the system. But because the amount of memory in modern systems has increased into the hundreds of gigabytes, it is now recognized that the amount of swap space that a system needs is a function of the memory workload running on that system. In the world of enhanced hardware and superior machines, the software enhancements and optimization for low end PC's is sometimes neglected. Hence, we propose a technique to overcome the lag created by the swap space by moving the content of the swap memory back to the RAM and a total boost is provided to the system. The technique is evaluated under various configurations and proves to solve the problem.

© 2019 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the INTERNATIONAL CONFERENCE ON RECENT TRENDS IN ADVANCED COMPUTING 2019.

Keywords: swap memory, swappiness, linux performance, swapping, virtual memory

^{*} Corresponding author. Tel.: +91-8825812496

E-mail address: shivam691999@gmail.com

1. Introduction

The low-end machines running on Linux based OS are having problems on performing normally after a heavy task. The computers start to hang and the UI becomes laggy, the applications take more time to load and the whole system slows down. The Linux Virtual Memory Management plays a crucial role in managing memory intensive tasks on a computer. When a heavy application is launched and the RAM is fully occupied, the Virtual Memory (Swap memory) starts filling. But when the applications are terminated and the RAM is available and is sufficient to accommodate the contents of the virtual memory, the back transition does not take place. This results in the crucial UI data (the content which was in RAM but had to be moved for the RAM to accommodate the new launched applications) to be stuck in the virtual memory. The content which was supposed to be in the RAM was retained in the swap memory (much slower than the RAM) which results in slow retraction of the content and the system starts to slow down.

As the content retrieval is slowed down, the processing speed is compromised. Also, because of reading content from virtual memory (which is on the hard disk) requires constant hard disk read/write function to run, the access speed of hard disk storage is reduced and hence, the processes that require hard disk read/write to function would now run much slowly and produce a large overhead in processing.

With our program running in the background the virtual memory management will enhance. This will be achieved by sensing the amount of available RAM and occupied swap memory and transfer of contents in swap to the RAM when the computer is ready to handle the load. And even after running huge programs the system will stabilize itself on its own.

2. Related Work

There has been very limited work done in the area of virtual memory management in Linux by using different methods to help the system gain the stability which is compromised by running memory intensive applications.

These research works focus on algorithms for managing virtual memory and on deciding the priority of the pages in the RAM to be virtualized. These research works deploy priority and scheduling algorithms to ensure stable performance of the computer.

Zhuanget al.[1] provided a set of recommendations for addressing the issues which can significantly affect the performance of the applications running in cgroups. Specifically, (1) memory is not reserved for cgroups (as with virtual machines); (2) both anonymous memory and page cache are part of memory limit and the former can evict the latter; (3) OS can steal page cache from any cgroups; (4) OS can swap any cgroups. While cgroups provides a decent mechanism to isolate the memory usage of regular cgroups, using it for application deployments requires special treatments to prevent applications running in cgroups from being affected in memory pressure scenarios. Their work discusses several of these scenarios and present solutions to address the problems.

Nakazawa et. al[2] presented a quantitative study of performance degradation of containerized workloads due to memory overcommitment and a technique to mitigate it. They focused on physical memory overcommitment, where the sum of the working set memory is larger than the physical memory. Their analysis showed that the throughput of a heavily loaded container decreased and varied when the memory is physically overcommitted. The reason is the working set memory of the heavily loaded container is swapped out even though it accesses memory more frequently than the lightly loaded ones.

Nakaike[3] worked on finding the major cause of performance degradation, which is page-cache reclamation operation which evicts frequently-accessed memory pages mapped to a file rather than swapping out rarely-accessed

anonymous pages. Their paper also confirms that a Linux virtual memory system can be enhanced to improve the performance of Java workloads that oversubscribe memory through an experiment which forces several file pages to persist in a physical memory. They investigated how they should enhance a virtual-memory system to improve the performance in a memory-oversubscribed environment. Their major finding is that the page-cache reclamation on a Linux virtual-memory system is a major cause of performance degradation when Java workloads oversubscribe memory.

Most researchers have fast computers which consist of more RAM [4] so they were unable to notice the absence of back-transition of the virtual memory contents back to the RAM. This issue was noticed by our team while working on a low-end computer a sudden performance drop was observed. On being intrigued by this instability in the system, this research was initiated and were successful in eliminating this problem.

3. Proposed Methodology

Linux based operating system's virtual memory management has compromised the stability of the computer. When a resource-hungry application which requires more RAM is run on a low-end device, the main memory (RAM) gets completely occupied and the virtual memory starts getting occupied. The UI is moved to the swap whereas the application being used by the user is loaded in the main memory (RAM). Through our methodology, the UI which is retained in the swap memory has to be allocated back inside the RAM without sabotaging the integrity of the processes running on the machine. This is achieved using a Linux command which seamlessly transfers the contents on the swap memory to the RAM. Also, the amount of available RAM is to be monitored so that the swapping is done only when there is enough RAM available on the PC to avoid any irregularities and data corruption. Also, a threshold of the amount of swap memory occupied has to be set so as to avoid unnecessary swapping process which may result in instability of the system and might crash. Once the swap is cleared, the system becomes stable and the applications work smooth. Also, the UI becomes more fluid and the programs take less time to start. The overall speedup is observed and existing performance-drop is cured.

From the flow diagram shown in **Fig.1** we observe that the available RAM is diagnosed and stored as R. Then the size of occupied swap memory is obtained and stored as S. We check if occupied swap is less than 100 MB, then the swapping operation is put on hold. Else if the occupied swap memory $\geq 100\text{MB}$, the next condition is checked. If $S > R$ then RAM will not be able to accommodate the swap contents, therefore, the swapping process is put to hold. Else if $R > S$ then the RAM is sufficient for the swapping to occur. The swapping is performed and the system stabilises. All the steps are repeated again.

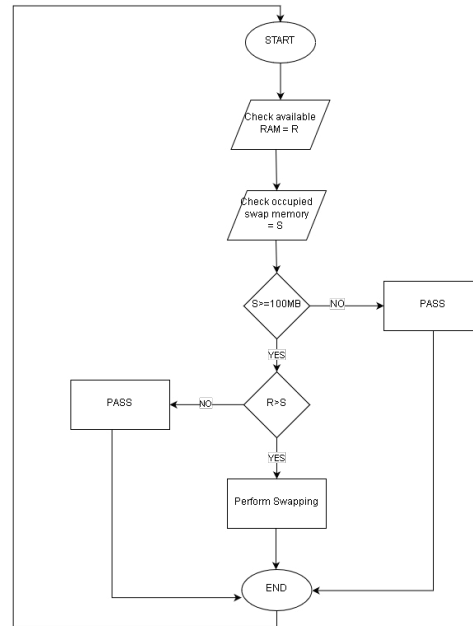


Fig.1 Flow Diagram

4. TESTING:

The program is written to run as a bash script and requires administrator privileges to run. The program also displays the following:

- Available RAM
- Total swap memory
- Used/Free swap memory
- Padded RAM (Available RAM – 100 MB) (For keeping some RAM free even after the swapping is completed so as not to clog the RAM completely).

The bash script is deployed using the command “*sudo bash name.sh*”. This script runs seamlessly in background on terminal.

For testing the program, the machine is completely loaded with applications to completely fill the RAM such that swap memory starts getting used. We fill the swap memory to a certain point and then close all the applications to empty the RAM, with and without the script running in the background. The PC is then given a time of 15 seconds to settle down the load and normalize the processing.

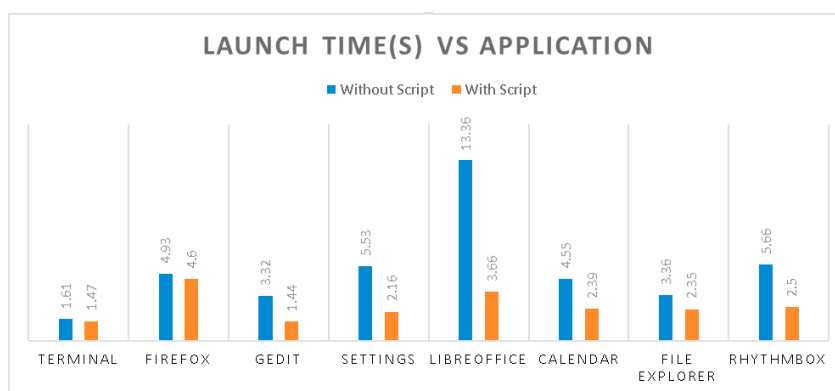
We ran the tests on several PC’s with different configurations and with different amount of swap memory utilized by the device and there by produced 4 different cases.

Also, Swappiness is the kernel parameter that defines how much (and how often) your Linux kernel will copy RAM contents to swap. This parameter's default value is “60” and it can take anything from “0” to “100”. The higher the value of the swappiness parameter, the more aggressively your kernel will swap.

4.1 CASE I: We used the system with Intel i3 6006u CPU, 4GB RAM, 1TB Hard disk configuration and 550MB swap memory utilised before draining RAM with default swappiness (i.e. default swappiness = 60).

Table 1. 4GB RAM with 550MB swap memory utilised before draining RAM with default swappiness(60).

Program	Without Script (in seconds)	With Script (in seconds)	Time Reduction in %
Terminal	1.61	1.47	+8.69%
Firefox	4.93	4.60	+6.69%
Gedit	3.32	1.44	+56.62%
Settings	5.53	2.16	+60.94%
LibreOffice	13.36	3.66	+72.6%
Calendar	4.55	2.39	+47.47%
File Explorer	3.36	2.35	+30.05%
Rhythmbox	5.66	2.50	+55.83%
Average Time Reduction			+42.36%

**Fig 2.** The clustered bar chart specifies Time (in Y- Axis) and launched application (in X- Axis) for **CASE 1**.

As shown in **Table 1** and **Fig 2**, we occupied the RAM completely so that the swap memory starts filling up. Then we launch the applications one by one and calculate their launch timings. It is to be taken into consideration that before launching an application, the previous application is terminated. In all the above programs, script running in the background yielded better results and the average time reduction for the launch and loading of the above applications was noticed.

4.2 CASE II: We used the system with Intel i3 6006u CPU, 4GB RAM, 1TB Hard disk configuration and 350MB swap memory utilised before draining RAM with default swappiness (i.e. default swappiness = 60).

Table 2. 4GB RAM with 350MB swap memory utilised before draining RAM with default swappiness(60).

Program	Without Script (in seconds)	With Script (in seconds)	Time Reduction
Terminal	1.76	1.33	+24.43%
Firefox	5.02	4.34	+13.5%
Gedit	1.36	1.23	+9.55%
Settings	2.12	1.99	+6.13%
LibreOffice	4.11	3.27	+20.43%
Calendar	2.36	1.4	+40.67%
File Explorer	2.22	2.09	+5.85%
Rhythmbox	2.6	2.22	+14.61%
Average Time Reduction			+16.89%

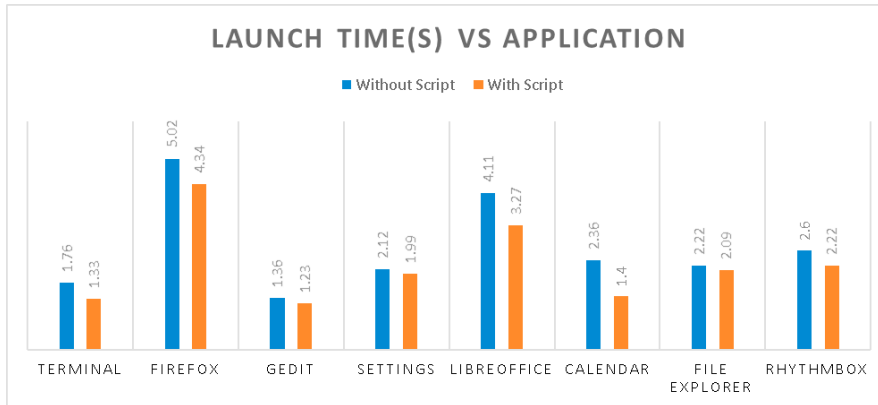


Fig 3.The clustered bar chart specifies Time (in Y- Axis) and launched application (in X- Axis) for **CASE 2**.

As shown in **Table 2** and **Fig 3**, we occupied the RAM completely so that the swap memory starts filling up. Then we launch the applications one by one and calculate their launch timings. It is to be taken into consideration that before launching an application, the previous application is terminated. In all the above programs, script running in the background yielded better results and the average time reduction for the launch and loading of the above applications was noticed.

4.3 CASE III: We used the system with Intel i3 6006u CPU, 2GB RAM, 1TB Hard disk configuration and 350MB swap memory utilised before draining RAM with default swappiness (i.e. default swappiness = 60).

Table 3. 2GB RAM with 350MB swap memory utilised before draining RAM with default swappiness(60).

Program	Without Script (in seconds)	With Script (in seconds)	Time Reduction
Terminal	5.53	2.9	+47.55%
Firefox	11.12	10.73	+3.5%
Gedit	5.52	2.93	+46.92%
Settings	8.14	6.34	+21.74%
LibreOffice	16.04	15.38	+4.11%
Calendar	11.58	3.07	+73.48%
File Explorer	3.57	2.62	+26.61%
Rhythmbox	4.79	3.88	+18.99%
Average Time Reduction			+30.36%

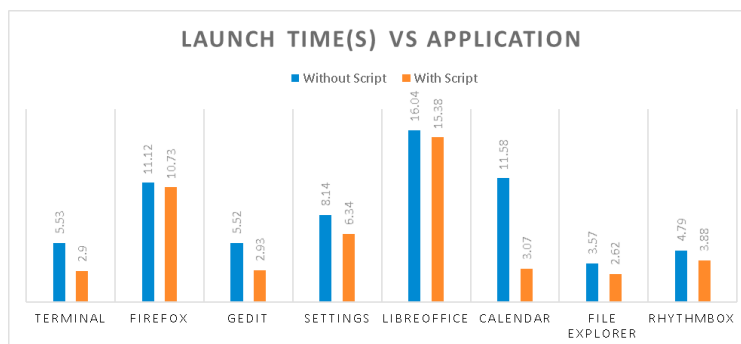


Fig 4.The clustered bar chart specifies Time (in Y- Axis) and launched application (in X- Axis) for **CASE 3**.

As shown in **Table 3** and **Fig 4**, we occupied the RAM completely so that the swap memory starts filling up. Then we launch the applications one by one and calculate their launch timings. It is to be taken into consideration that before launching an application, the previous application is terminated. In all the above programs, script running in the background yielded better results and the average time reduction for the launch and loading of the above applications was noticed.

4.4 CASE IV: We used the system with Intel i3 6006u CPU, 4GB RAM, 1TB Hard disk configuration and 550MB swap memory utilised before draining RAM with default swappiness (i.e. default swappiness = 10).

Table 4. 4GB RAM with 550MB swap memory utilised before draining RAM with swappiness set to 10.

Program	Without Script and swappiness = 10 (in seconds)	With Script (in seconds)	Time Reduction
Terminal	1.83	1.47	+19.6%
Firefox	5.34	4.60	+13.85%
Gedit	1.50	1.44	+4%
Settings	2.31	2.16	+6.4%
LibreOffice	11.13	3.66	+67.11%
Calendar	2.62	2.39	+8.7%
File Explorer	2.53	2.35	+7.11%
Rhythmbox	3.17	2.50	+21.13%
Average Time Reduction			+18.48%

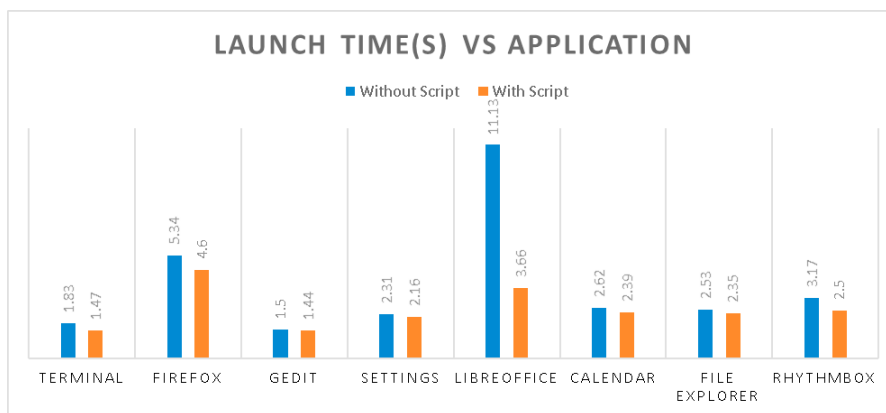


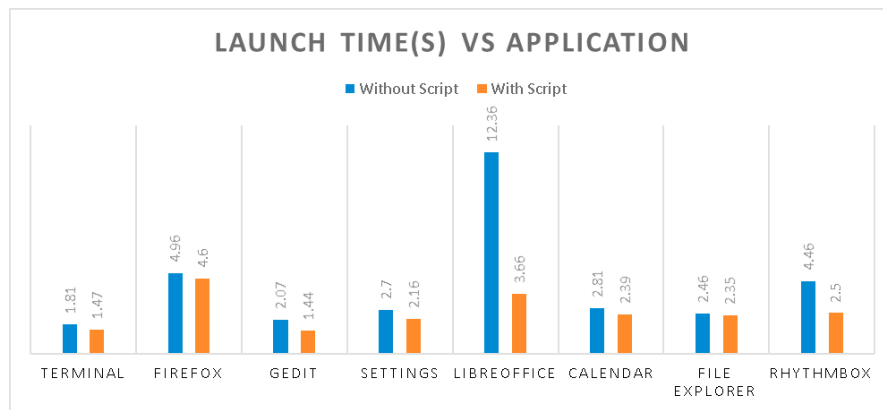
Fig 5. The clustered bar chart specifies Time (in Y- Axis) and launched application (in X- Axis) for **CASE4**.

As shown in **Table 4** and **Fig 5**, we occupied the RAM completely so that the swap memory starts filling up. Then we launch the applications one by one and calculate their launch timings. It is to be taken into consideration that before launching an application, the previous application is terminated. In all the above programs, script running in the background yielded better results and the average time reduction for the launch and loading of the above applications was noticed.

4.5 CASE V: We used the system with Intel i3 6006u CPU, 4GB RAM, 1TB Hard disk configuration and 550MB swap memory utilised before draining RAM with default swappiness (i.e. default swappiness = 100).

Table 5. 4GB RAM with 550MB swap memory utilised before draining RAM with swappiness set to 100.

Program	Without Script and swappiness = 100 (in seconds)	With Script (in seconds)	Time Reduction
Terminal	1.81	1.47	+18.78%
Firefox	4.96	4.60	+7.25%
Gedit	2.07	1.44	+30.43%
Settings	2.70	2.16	+20%
LibreOffice	12.36	3.66	+70.38%
Calendar	2.81	2.39	+14.94%
File Explorer	2.46	2.35	+4.47%
Rhythmbox	4.46	2.50	+43.94%
Average Time Reduction			+26.27%

**Fig 6.** The clustered bar chart specifies Time (in Y- Axis) and launched application (in X- Axis) for CASE5.

As shown in **Table 5** and **Fig 6**, we occupied the RAM completely so that the swap memory starts filling up. Then we launch the applications one by one and calculate their launch timings. It is to be taken into consideration that before launching an application, the previous application is terminated. In all the above programs, script running in the background yielded better results and the average time reduction for the launch and loading of the above applications was noticed.

5. Analysis:

Using the data from the tests done on different devices with different configurations and under different circumstances, a huge reduction in time to open different applications was observed. The most noticeable change was observed when the swap memory was occupied more than the other cases (i.e. in Case I) because of higher swap memory occupancy was making the system more unstable but the script stabilised. Through all the cases, it was keenly observed that the applications took less time to launch with the script running in the background. The User Interface also seemed to be more fluid and no lag was experienced with the script. Without the script, a noticeable lag in the User Interface along with slow start-up of applications was noticed.

Another thing which was observed in all the cases was, that after a fresh start of Operating System the RAM utilised was approximately 1.3 GB. After running memory intensive applications and filling of swap memory, then

closing all the open applications, did not affect the used swap memory. Also, the used RAM was found to be 782MB. This concludes that the fresh User Interface which was earlier 1.3GB has a huge chunk of almost 500 MB in swap memory and hence, generates a significant overhead.

Our method and implementation clearly enhanced the performance of the systems. With our script running in the background, the system ran smoothly despite their being an immense pressure on the system's main memory.

6. Conclusion and Future Scope

As in the results, it is clearly observed that our script not only boosted the performance of the computer but also made the OS more stable. The UI was noticed to be more fluid and smoother, the applications took lesser time to load and launch and the overall processing of the computer was increased. The Linux virtual memory management was incapable to bring the computer back to its stable state and hence with our smart swapping script running in the background, better results were obtained.

The program is written as a bash script and can be deployed using `sudo bash name.sh`. This will require administrative privileges to run. This shell script can be deployed as a thread with auto-startup privileges to run seamless in background. This script can be bundled up with the Linux update packages and installed on all the Linux devices through over-the-air updates. The PC's with low RAM which continuously hustle after running memory intensive applications will get to see the major difference. The servers running on the cloud which process a huge amount of data will perform better and process faster. The faster computers with more RAM can also enhance their performance while they run RAM hungry applications. Our research might prove to be a vital solution to all the performance issues faced by the users.

References:

- [1] Zhenyun Zhuang, Cuong Tran, Jerry Weng, Haricharan Ramachandra and Badri Sridharan (2017) "Taming Memory Related Performance Pitfalls in Linux Cgroups". International Conference on Computing, Networking and Communications (ICNC): Cloud Computing and Big Data
- [2] Rina Nakazawa, Kazunori Ogata, Tamiya Onodera and SeetharamiSeelam (2017) "Taming Performance Degradation of Containers in the Case of Extreme Memory Overcommitment". IEEE 10th International Conference on Cloud Computing.
- [3] Takuya Nakaike, Yohei Ueda, Takanori Ueda and Moriyoshi Ohara (2017) "Performance Characteristics of Linux for Java Workloads Oversubscribing Memory". Proceedings of the 34th Annual Meeting of the Japan Society for Software Science (2017)
- [4] https://access.redhat.com/documentation/en-US/Red_Hat_Enterprise_Linux/4/html/Introduction_To_System_Administration/s1-memory-concepts.html